



Master 2 – Modélisations Statistiques Economiques et  
Financières Data Science

**Projet API / Webscrapping**

Projet réalisé par :

**COLIN Léo**

**BENARD Antonin**

**QUINTIN DE KERCADIO Pierre**

# 1 Introduction et Problématique

Le projet **CV-Optimizer** s'inscrit dans une démarche d'automatisation intelligente du processus de recherche d'emploi. Face à un marché du travail fragmenté et à la multiplication des plateformes, l'adaptation manuelle d'un CV pour chaque offre est une tâche chronophage. Notre solution propose un pipeline complet capable d'agréger des offres multi-sources, d'évaluer la pertinence sémantique entre un profil et un poste, et de générer des conseils personnalisés via des modèles de langage de pointe.

## 2 Méthodologie et Architecture Système

### 2.1 Collecte des données

La collecte des données repose sur une approche hybride afin de maximiser la couverture des offres. Nous exploitons l'API officielle de France Travail pour une récupération structurée, complétée par un travail de web scraping sur HelloWork et Welcome to the Jungle via Selenium 4.39. Ce choix technique a été fait pour gérer le contenu dynamique chargé en JavaScript.

Nous avons choisi d'exclure les sites de carrières de notre périmètre de scrapping, le volume d'offres y étant souvent trop limité pour justifier le développement d'un scraper dédié, et les annonces généralement déjà agrégées sur les plateformes majeures que nous avons ciblé.

Afin de garantir l'intégrité de la base de données, un identifiant unique est attribué à chaque offre. Lorsque le site d'origine ne fournit pas d'ID, un identifiant unique est généré par notre système. Ce mécanisme permet d'éviter strictement l'insertion de doublons et d'assurer la fiabilité et la qualité de notre solution. Plus précisément, pour les sources ne fournissant pas directement d'identifiant exploitable, nous en générons un en concaténant les valeurs normalisées de l'entreprise, du titre et du lieu de travail, puis appliquons un hachage SHA-256 sur cette chaîne concaténée pour produire un identifiant unique de 64 caractères hexadécimaux. Cette méthode garantit qu'une même offre d'emploi génère toujours le même identifiant, facilitant ainsi la détection des doublons. Le scraping d'une offre d'emploi n'est effectué qu'après vérification de son absence en base de données. De plus, lorsque l'utilisateur renseigne plusieurs mots-clés similaires au moment de lancer un webscraping, cas fréquent pour une recherche ciblée dans un domaine spécifique, le

système vérifie également qu’une même offre n’a pas déjà été scrapée lors de la même exécution avec un mot-clé différent. Bien que l’utilisation des identifiants fournis par les sites sources nous expose à un risque en cas de modification de leur système d’identification, cette approche permet un gain de temps considérable durant le processus de scraping.

## 2.2 Traitement et Stockage

La persistance des données est assurée par une base PostgreSQL 16. Le choix de cet outil est stratégique : l’extension pgvector permet de stocker et d’interroger des vecteurs directement en SQL. L’interaction avec la base est facilitée par l’ORM SQLAlchemy 2.0, assurant une gestion robuste des sessions Python.

Pour le traitement des documents, notamment des CV, nous utilisons PyMuPDF pour extraire le texte brut. Ce processus est complété par une étape de normalisation Unicode, indispensable pour garantir la cohérence des caractères avant l’étape de vectorisation. Aussi, nous exploitons la bibliothèque BeautifulSoup pour nettoyer les descriptions d’offres d’emploi en supprimant les balises HTML résiduelles issues du web scraping, assurant ainsi la pureté du texte et la qualité du matching sémantique final.

## 2.3 Matching Sémantique

Le moteur de recommandation exploite les avancées du traitement du langage naturel (NLP) pour transformer des textes bruts en données exploitables mathématiquement. L’objectif est de capturer l’intention derrière une offre d’emploi plutôt que de se limiter à une simple correspondance de mots-clés. Le processus s’appuie sur deux piliers techniques :

- **La vectorisation** : Le modèle paraphrase-multilingual-MiniLM-L12-v2 convertit les descriptions d’offres et les CV en vecteurs numériques de 384 dimensions. Ce modèle de sentence-BERT, entraîné sur plus de 50 langues, utilise une architecture basée sur des transformers pour capturer le sens sémantique profond des textes plutôt que de simplement comparer des mots-clés. Cette approche permet de quantifier la proximité entre un profil et un poste : plus les vecteurs sont proches dans l’espace mathématique, plus la cohérence sémantique est élevée. Le modèle est particulièrement adapté à notre cas d’usage car il est optimisé pour détecter les paraphrases et les similarités sémantiques, permettant ainsi de faire correspondre des compétences formulées différemment mais ayant le même sens (par exemple,

"développement web" et "création de sites internet").

- **Le matching hybride** : La recherche s'effectue via une requête SQL permettant de calculer les similarités cosinus entre le CV vectorisé et les embeddings stockés en base de données. Pour garantir une sélection réaliste, ce score sémantique est croisé avec des critères logistiques indispensables tels que la localisation, le type de contrat et la date de publication.

## 2.4 Génération de Conseils Personnalisés

Pour finir, nous intégrons le modèle de langage Qwen/Qwen2.5-7B-Instruct, un modèle open source accessible via l'API Hugging Face. Ce dernier intervient pour apporter une analyse qualitative indispensable à l'optimisation d'une candidature. En identifiant les écarts entre le profil du candidat et les exigences de l'offre, le modèle génère des recommandations concrètes et actionnables. Il se concentre sur trois axes principaux : l'identification des compétences techniques manquantes, l'intégration de mots-clés stratégiques alignés avec l'offre, et la proposition d'accroches personnalisées. Cette étape est cruciale pour permettre au candidat de franchir les filtres de sélection automatisés (ATS) et maximiser ses chances d'être retenu par les recruteurs. Le prompt structuré envoyé au LLM suit une approche systématique pour garantir des conseils pertinents et exploitables :

```
{
  "role": "system",
  "content": "Tu es un expert en recrutement français. Ta mission
est d'aider un candidat à adapter son CV."
},
{
  "role": "user",
  "content": f""""Compare mon CV avec cette offre.
OFFRE : {offer.title}
DESCRIPTION : {offer.description[:1500]}

MON CV : {cv_text[:1500]}

Donne une réponse structurée :
1. Points forts (2-3 points).
```

2. Compétences manquantes (mots-clés ou technologies).
  3. Un conseil pour une accroche brève (2-3 phrases) en haut du CV qui permette d'aligner au maximum le profil avec l'offre, tout en respectant les compétences réelles du candidat."""
- }

## 3 Description du Jeu de Données

La base de données JobOffers centralise l'ensemble des informations collectées. Chaque entrée est conçue pour être exploitée à la fois à des fins d'analyse ou par des modèles d'IA.

### 3.1 Structure et Métadonnées

Les données sont organisées selon trois dimensions principales :

- **Informations d'Identification** : Identifiants uniques (ID), sources d'origine, URLs et stockage de l'offre brute au format JSON pour archivage.
- **Attributs Métiers** : Titre du poste, entreprise, localisation géographique, type de contrat et niveau d'expérience requis. Ces champs permettent un filtrage précis post-matching.
- **Données Temporelles** : Dates de création, d'actualisation et d'insertion en base.

Les champs d'attributs métiers permettent un filtrage précis post-matching.

### 3.2 Recherche Vectorielle

L'atout du jeu de données réside dans son enrichissement systématique. Chaque offre insérée passe par un processeur d'embeddings qui génère sa signature numérique en temps réel (traitement par lots). Cette structure permet de passer d'une base de données textuelle classique à une base de connaissance sémantique prête pour la recherche vectorielle. Chaque offre possède donc un vecteur associé dans notre base.

## 4 Présentation de l'API

### 4.1 Architecture générale

L'API CV-Optimizer est construite avec FastAPI et suit une architecture modulaire, organisée autour de plusieurs endpoints spécialisés. Elle orchestre la collecte, le traitement, la vectorisation et le matching d'offres d'emploi avec des CV, tout en intégrant des fonctionnalités avancées de recommandation via LLM. L'API interagit avec une base de données PostgreSQL (via SQLAlchemy) et s'appuie sur des services de scraping, de NLP et d'indexation.

L'architecture se compose de plusieurs modules :

- **Endpoints** : chaque fonctionnalité principale possède son propre endpoint (matching, conseils LLM, gestion des offres, etc.).
- **Services** : la logique métier (scraping, vectorisation, matching, etc.) est externalisée dans le dossier `services`.
- **Base de données** : toutes les offres et résultats sont stockés dans PostgreSQL.
- **Sécurité** : certains endpoints sont protégés par un secret pour contrôler à terme qui pourra collecter des offres (une fois la base hébergée sur un Cloud).

Le schéma de routage principal (voir `router.py`) regroupe tous les endpoints sous le préfixe `/api`, à l'exception du endpoint *Health*.

### 4.2 Détail des endpoints

#### 4.2.1 Endpoints de santé (`/` et `/health`)

- `GET /health` : Vérifie que l'API répond correctement.
- `GET /api/status` : Vérifie que l'API répond correctement et que la connexion à la base de données peut s'effectuer.

#### 4.2.2 Matching CV/Offres (`/api/match`)

- `POST /api/match` : Permet d'uploader un CV (PDF) et retourne les offres d'emploi les plus pertinentes selon un matching sémantique. Possibilité de filtrer par localisation, type de contrat, expérience, etc.

### 4.2.3 Conseils LLM personnalisés (/api/advice)

- `POST /api/advice` : Permet d'uploader un CV et de demander des conseils personnalisés générés par un LLM (Qwen 2.5) pour optimiser le CV en fonction d'une offre ciblée (via son ID).

### 4.2.4 Consultation des offres (/api/jobs)

- `GET /api/jobs` : Liste les offres d'emploi présentes en base, avec de nombreux filtres (localisation, type de contrat, expérience, source, mots-clés, date, etc.) et supporte la pagination.

### 4.2.5 Endpoints d'administration (/api/admin/...)

Protégés par clé API (header `X-API-Key`), réservés à la maintenance et à la collecte :

- `POST /api/admin/collect` : Lance la collecte de nouvelles offres (API France Travail, scraping HelloWork et Welcome to the Jungle), insère en base et vectorise automatiquement.
- `POST /api/admin/reindex` : Régénère tous les embeddings pour les offres en base (utile en cas de changement de modèle ou de correction de bugs).
- `GET /api/admin/stats` : Retourne des statistiques sur la base (nombre d'offres, répartition par source, taux d'indexation, offre la plus récente, etc.).

Cette organisation permet une séparation claire entre les fonctionnalités utilisateurs (matching, conseils, consultation) et les opérations d'administration, en assurant la sécurité et la scalabilité de l'API.

## 5 Résultats

Pour conclure sur les résultats obtenus, nous pouvons observer que le modèle présente des performances globalement satisfaisantes, avec des scores de pertinence cohérents entre les offres et les CV lors des phases de test. Le système parvient notamment à établir des correspondances pertinentes entre des profils et des offres même lorsque les intitulés diffèrent (par exemple « Développeur Python » et « Ingénieur Back-end »), grâce à la proximité sémantique de leurs embeddings respectifs.

Par ailleurs, le modèle Qwen-2.5-7B apporte une réelle valeur ajoutée sur le plan

pédagogique. En identifiant les compétences présentes dans l’offre mais absentes du CV, le LLM est en mesure de générer des recommandations d’adaptation pertinentes. Ces suggestions permettent au candidat d’optimiser stratégiquement son CV pour chaque offre ciblée, augmentant ainsi significativement ses chances de franchir les systèmes de tri automatique utilisés par les entreprises et d’attirer l’attention des recruteurs.

## 6 Prochaines étapes

- **Amélioration du matching sémantique** : Prétraiter les descriptions d’offres pour en extraire uniquement les sections pertinentes (compétences requises, missions, profil recherché) avant la vectorisation, en écartant les informations parasites telles que la présentation générale de l’entreprise ou les avantages non techniques. Cette approche permettrait de réduire le bruit dans les embeddings et d’améliorer la précision des recommandations en se concentrant sur les critères réellement discriminants pour le matching candidat-poste.
- **Intégration d’un module OCR** : Ajouter une couche de reconnaissance optique de caractères (Tesseract ou EasyOCR) pour permettre l’extraction de texte depuis des CV scannés ou des PDF non natifs, élargissant ainsi la compatibilité avec différents formats de documents.
- **Développement d’une interface utilisateur** : Concevoir un front-end intuitif (React ou Streamlit) permettant aux utilisateurs de télécharger leur CV, visualiser les offres recommandées et consulter les conseils personnalisés sans interaction avec la ligne de commande.
- **Migration de la base de données vers le Cloud** : Déployer la base de données PostgreSQL sur une infrastructure cloud centralisée (AWS) afin que tous les utilisateurs accèdent à une base commune et enrichie. Actuellement, chaque instance locale du projet génère sa propre base de données isolée, obligeant chaque nouvel utilisateur à repartir de zéro. Une base centralisée permettrait de mutualiser les offres d’emploi collectées et d’offrir immédiatement un catalogue étoffé à tous les utilisateurs.
- **Mise en place d’alertes hebdomadaires automatisées** : Implémenter un système de notifications par email permettant aux utilisateurs de recevoir régulièrement les nouvelles offres correspondant à leur profil, assurant ainsi une veille active et continue du marché de l’emploi sans nécessiter de connexion manuelle à la plateforme.